

ViewTree: On-Device Spatial Intelligence via Confidence-Guided Camera Walks over Explicit Scene Memory

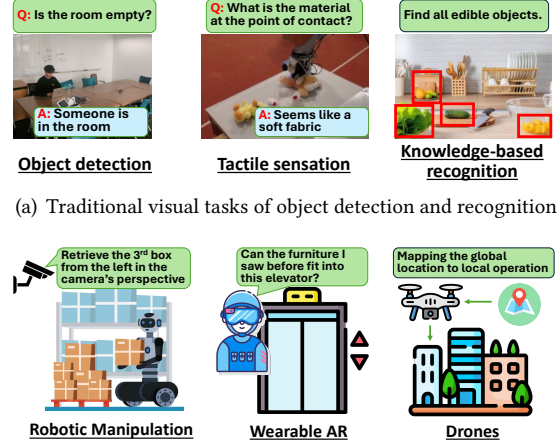
ABSTRACT

Spatial questions – how far apart two objects are, which direction one lies from another, how large a room is – are increasingly asked of vision-language models (VLMs) on mobile and embedded devices. A VLM that passively reads a fixed set of video frames integrates cross-view geometry poorly, and feeding it more frames quickly exhausts on-device latency, memory, and energy budgets. This paper presents ViewTree, an on-device spatial reasoning system that gives the VLM an *explicit scene memory* – a 3D reconstruction built once from the captured video – and lets it take *confidence-guided camera walks* over that memory: short sequences of physically plausible virtual-camera movements in which a learned confidence signal decides when to explore, when to branch into alternative paths, and when to stop and answer. Every acquired view is a re-projection of the same stored geometry, so exploration does not accumulate the errors of generative world models. On the VSI-Bench spatial reasoning benchmark, ViewTree lifts a 7B backbone from 0.313 to 0.530 average accuracy and adds 2.1 points over a baseline fine-tuned on the same data, with the largest gains on perspective-dependent questions. On NVIDIA Jetson AGX Orin, confidence gating answers 71% of questions faster than a single 16-frame VLM call, and the average cost of 24.5 s and 1.1 kJ per question is 1.9× cheaper than a non-adaptive view tree.

1 INTRODUCTION

Many mobile and embedded devices nowadays use visual streams as the input to vision-language models (VLMs) to perceive the physical world and guide device actions [9, 22]. Instead of using cloud AI services, executing VLMs locally protects data privacy and avoids possible network disruptions, both of which are important to interactive applications such as robotic manipulation [4, 15], navigation [25], AR/VR assistants [1, 19] and mobile video analytics [31].

Being different from traditional appearance-based visual tasks such as object detection [7, 20] and semantic segmentation [16, 20] shown in Figure 1(a), these emerging applications build on the ability of **spatial intelligence**, which comprehends the 3D physical properties of the environment and geometric relations (e.g., distance, orientation, and relative positioning) between objects in different views. For example in Figure 1(b), to identify the correct box to retrieve,



(b) Emerging visual tasks based on spatial intelligence

Figure 1: Different VLM-based tasks on mobile and embedded devices

a robot translates the allocentric input command into ego-centric actions, requiring correct understanding of the spatial relationships across the camera’s view and robot’s view. Similarly, a wearable AR assistant estimates the object’s 3D dimensions to determine whether it fits into the elevator, and a search-and-rescue drone builds a 3D map of an unknown area to report the survivor’s location relative to the landmarks.

In these tasks, the spatial information needed for the task is usually outside the current view and distributed across several video frames. Current VLMs have been proved to be unreliable on integrating spatial information from multiple viewpoints [11, 26, 33, 36], because they perceive a 3D scene only through separated 2D images and cannot properly choose what to observe next. More specifically, each image is encoded according to the corresponding camera view, but the same object may appear at a different location/scale or being occluded in different frames. To combine these frames, the VLMs need to precisely estimate how the camera moves across frames, match the same object across views, place them in a unified 3D space, and memorize the objects’ relationships in context. This is difficult for most of today’s VLMs, especially small on-device VLMs [33, 36]. Moreover, the VLM can only perceive the frames given as input but cannot control the camera movement or request new views, even if important information is missing. Simply adding more frames does not help because these frames still

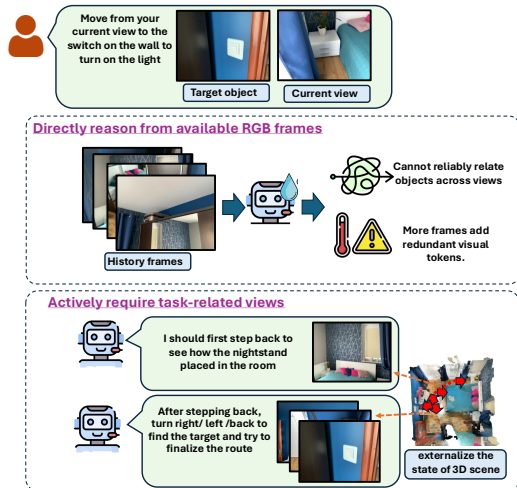


Figure 2: ViewTree actively requests task-relevant views instead of perceiving redundant frames

need to be correctly aligned and combined, and redundant or irrelevant views may distract the VLM and cause extra errors [28].

An intuitive solution is to explicitly provide 3D information to the VLM. Existing methods inject 3D supervision during VLM training [6, 29], or construct bird’s-eye-view (BEV) and semantic maps from observed frames [23, 28]. Although these approaches can improve spatial grounding, the fixed global representation they provide may not be always suitable for a small on-device VLM, because using dense geometry contains substantial task-irrelevant information and using a sparse map must decide in advance which objects and relationships to preserve. In both cases, the VLM passively receives a predetermined representation that may not contain all the spatial information required in reasoning.

Instead, a better alternative is to externalize the state of 3D scene and allow the VLM to *actively acquire* the necessary spatial information in task-relevant views. More specifically, a VLM can use a *world model* to generate new views [5, 34, 37, 41], which help reveal an occluded object or show a spatial relationship from a viewpoint that is easier for the VLM to perceive. For example, as shown in Figure 2, the VLM first requests a view from further back to locate the nightstand and then requests a view facing the nearby wall to find the light switch. In this way, it uses only a small number of task-relevant views, rather than reasoning over all the available frames that may be irrelevant and hence confusing.

However, the generated view is only a prediction from the world model, rather than an actual observation from the captured scene. Especially when the prediction corresponds to large camera movement, it may lose physical consistency and largely deviate from the actual scene [14], as shown in

Figure 3. Leveraging multiple predictions from the world model can help, but adds substantial compute cost to mobile devices. To address this issue, our basic rationale is to use a geometry grounded 3D reconstruction model, such as VGGT [30], to reconstruct the input video scenes as a set of 3D structures (e.g., 3D point clouds or depth maps) with estimated camera poses. Afterwards, when the VLM requests a camera movement, the system can virtually move to the requested camera pose and project the available 3D structures to a new 2D view. In this way, the rendered view can ensure to preserve the geometry and other physical properties of the original scene. Moreover, this also prevents errors from accumulating over consecutive movements: a world model must feed its own prediction back as the next input, so its errors compound, whereas every rendered view is re-projected from the same stored reconstruction (Figure 3).

Based on this rationale, in this paper we present ViewTree, an on-device system that gives the VLM an *explicit scene memory* – the reconstruction and the views rendered from it – and lets the VLM take *confidence-guided camera walks* through this memory: short sequences of virtual camera movements in which a learned confidence estimate decides where to walk, whether to branch, and when to stop and answer. To limit the damage of a wrong decision propagating along a single reasoning path, we allow a VLM to adaptively explore multiple camera paths in the reconstructed 3D scenes. While an easy question can be answered from the current view or after one camera movement, a more difficult question can explore additional paths. The main challenge is how to explore multiple camera paths without introducing unnecessary computations on resource-constrained mobile devices. ViewTree adds a learned confidence head that predicts whether each partial path can eventually produce the correct answer. When the current answer has low confidence and several camera movements appear useful, the VLM can create a small number of paths from the same reasoning history. These paths reason independently, so that an incorrect decision in one path does not affect the others. After each movement, ViewTree retains paths with high confidence and removes paths with low confidence. If several high-confidence paths produce different answers, the system keeps them and requests another view that can help resolve the disagreement. Observations from different paths are combined only when they provide complementary information.

With the available set of camera paths, at each reasoning step the VLM selects one of four operations: MOVE, BRANCH, FUSE, or STOP. MOVE changes the virtual camera position and returns an encoded view with its camera pose. BRANCH forks the current reasoning history into a few paths that

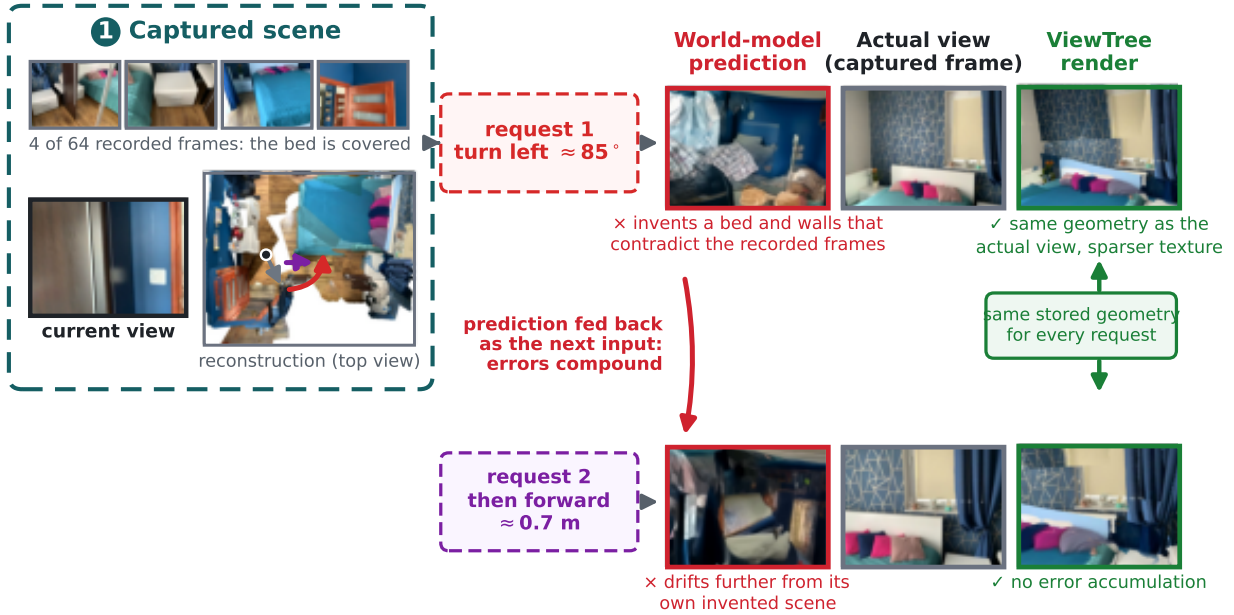


Figure 3: World-model view prediction versus ViewTree’s reconstruct-then-render, on a real captured scene. From the current view, the VLM requests “turn left $\approx 85^\circ$ ” and then “move forward ≈ 0.7 m”; both requested views were actually captured in the walkthrough, so the correct outcome of each movement is known. The world model (Stable Virtual Camera [40], the world model used by MindJourney [34]) predicts a bed and walls that contradict the recorded frames, and drifts further when its own prediction is fed back as the input for the second movement. ViewTree instead renders both requested views by re-projecting one reconstruction built from all frames: each view stays consistent with the actual scene, and errors cannot accumulate across movements.

continue independently from the same context. FUSE combines the views retained by different paths into one context to produce a single answer. STOP ends a path and commits its answer together with a confidence estimate. Figure 4 illustrates the four operations. In this way, the VLM receives only the view requested by its current reasoning, rather than processing the complete 3D reconstruction.

To run efficiently on mobile and embedded devices, ViewTree reuses computation shared by different camera paths. Camera movements from different paths are rendered and encoded in batches, and the paths share the VLM’s prefix KV cache before they separate. A device-aware scheduler then adjusts the number and length of camera paths according to the available latency, energy, memory, and thermal budget. In this way, the VLM decides which paths are useful, and the scheduler determines how many useful paths the device can support.

We implemented ViewTree with PyTorch and HuggingFace Transformers, training on a server with 8×NVIDIA H100 GPUs and deploying the complete inference pipeline on an

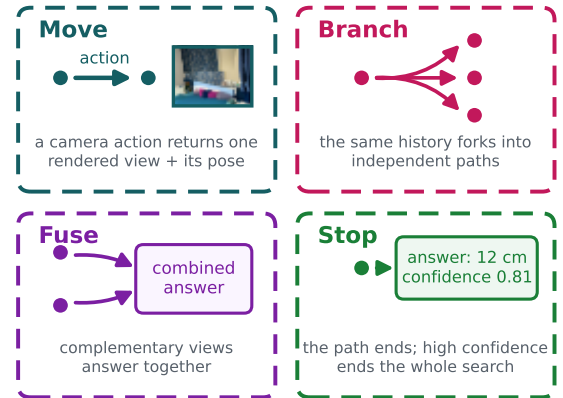


Figure 4: The four reasoning operations of ViewTree.

NVIDIA Jetson AGX Orin (64 GB, JetPack 6). We evaluated ViewTree with Qwen2.5-VL backbones of 3B, 7B, and 32B parameters [3] on four spatial reasoning benchmarks: VSI-Bench [33], STI-Bench [17], VSTI-Bench [10], and OST-Bench [18].

From the experiment results, we have the following conclusions:

- ViewTree improves spatial reasoning accuracy on VSI-Bench from 0.313 (direct 16-frame input) to 0.530 with the same 7B backbone. Against the strongest baseline fine-tuned on the same training data, ViewTree’s adaptive exploration adds 2.1 points on average, and the gains concentrate exactly on the questions where a new viewpoint changes the visible geometry: +6–7 points on relative direction and +5 points on relative distance.
- Adaptive exploration is affordable on-device. On Jetson AGX Orin, ViewTree answers 71% of the questions directly after a cheap sufficiency check in 9.6 s – faster than a single 16-frame VLM call (11.7 s) – and averages 24.5 s and 1.1 kJ per question, 1.9× cheaper than a non-adaptive tree that explores rendered views for every question (46.3 s, 2.1 kJ).
- Adaptivity is also necessary for accuracy: exposing rendered views to every question unconditionally helps a 7B backbone (+2.9 points) but *reduces* a 32B backbone’s accuracy by 3.2 points, and unconditional multi-path exploration costs over 12× a direct answer. ViewTree’s learned confidence-based control decides per question whether extra views pay off.

2 BACKGROUND AND MOTIVATION

2.1 Spatial Reasoning from Fixed Views

Existing 3D question-answering benchmarks ask a VLM to reason from the current image, a set of history frames, or a video clip [2, 21, 22]. This input is sufficient when the queried objects are visible in one view, but perspective taking, occlusion, and spatial memory often require information from a camera position that was never given [33, 36].

To quantify this gap beyond the examples in Section 1, we measured Qwen2.5-VL-7B on the held-out half of VSI-Bench (2,557 questions; Section 7). With 16 uniformly sampled frames, the model averages only 0.31 over the ten task types, and collapses on exactly the tasks that need cross-view geometry: 0.09 relative accuracy on absolute distance and 0.29 on appearance order. Two natural remedies fail for different, instructive reasons. *Fine-tuning does not fix it*: training the same backbone on the benchmark’s own training distribution moves the average by only +1.4 points (0.327), and adding answer-reward reinforcement learning on top changes nothing – the bottleneck is the input, not the weights. *More frames do not fix it either*: as Figure 5 shows on Jetson AGX Orin, per-question latency grows superlinearly from 12.3 s at 16 frames to 300 s at 384, the process runs out of the

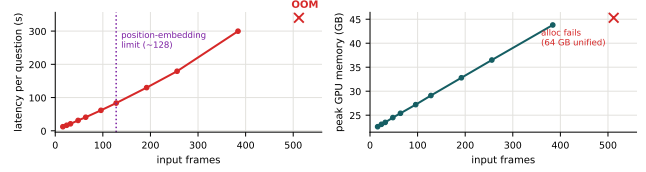


Figure 5: Scaling the frame count on Jetson AGX Orin (7B backbone, measured): latency grows superlinearly and memory is exhausted at 512 frames, while the position-embedding limit binds near 128 frames.

64GB unified memory at 512 frames, and the model’s position embeddings bind even earlier, near 128 frames. Meanwhile consecutive frames of a walkthrough are near-duplicates, so the added tokens mostly repeat evidence the model already failed to use. Other work converts the history frames into a fixed bird’s-eye-view or semantic map [23, 28]; such maps align information across views, but they must decide in advance what to include and cannot supply a missing view during reasoning.

These measurements motivate ViewTree to change the input rather than enlarge it: the VLM should be able to request the specific missing view, instead of receiving ever more fixed frames or one fixed map.

2.2 Generating or Rendering New Views

Recent work allows a VLM to request new views from a generative world model [5, 34, 37, 41]: the VLM chooses a camera movement and the world model synthesizes the requested viewpoint. The generated image, however, is a prediction rather than an observation; large camera movements produce views that are geometrically inconsistent with the captured scene [14], and every imagined view costs a full generative-model invocation on a device that is already running the VLM. Reconstructing the observed scene and rendering from it avoids both problems: all rendered views refer to the same stored geometry, and rendering is nearly free at the system level – on Jetson AGX Orin a rendered view costs 0.12 s, two orders of magnitude below the 4–5 s VLM call that must read it. The real question is therefore not how to produce views, but *which few views the VLM should be asked to read*.

Our measurements show that naive exposure of the reconstruction fails in two concrete ways. First, viewpoint proposals must be physically grounded: a naive proposer that simply orbits the reconstruction placed *none* of its cameras inside the room on held-out scenes – every view looked down through the ceiling from outside (Figure 6) – and even among human-reachable poses, roughly a quarter render too little supported geometry and must be rejected. Second, unconditionally attaching a fixed set of rendered views to

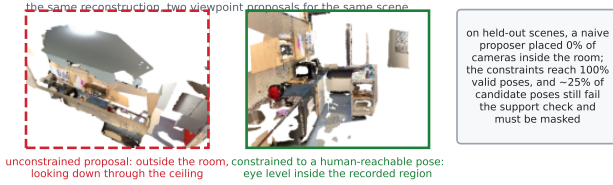


Figure 6: Two viewpoint proposals on the same reconstruction. Unconstrained proposals view the scene from outside, through the ceiling; ViewTree restricts poses to positions a person could occupy and masks poses without geometric support.

every question helps on average but hurts where the renders are irrelevant: with the 7B backbone it adds +2.9 points overall yet *reduces* accuracy on object counting and room size once the model is trained to attend to renders, and with a 32B backbone the same fixed recipe is 3.2 points *below* frames-only prompting – a stronger model is distracted more by views it did not ask for.

These observations motivate the two design choices of Section 4: candidate views are constrained to physically plausible, renderable poses, and they are acquired selectively – only when the current question needs them – rather than attached wholesale.

2.3 Single and Multiple Reasoning Paths

Existing methods allow a VLM to move a virtual camera through a reconstructed 3D scene [38, 39], but follow a single camera path. Figure 7 shows why this is brittle, on a real held-out question. Asked for the size of an exhaust fan, the model’s two best first movements are nearly indistinguishable in confidence (0.700 vs. 0.697), yet the marginally more confident one answers 20 cm – 67% away from the ground truth of 12 – while the runner-up answers 14. A single-path search must commit at exactly the moment when confidence carries the least information; here it would commit wrongly. Retaining both paths defers the decision: one movement later, every continuation of *both* paths agrees on 14, and the search stops with an answer 17% from the ground truth. This pattern is not rare: on the held-out evaluation, 209 questions reach agreement only at depth two or three, and on those questions the direct answer was correct less than half the time.

The opposite extreme – expanding several paths at every step, as generic multimodal tree search does [32], or rendering fixed camera grids in advance [27] – is unaffordable on device. Each additional path pays view rendering, visual encoding, VLM prefill, and memory for its reasoning history: fully expanding three movements per step to depth three costs 21 VLM calls, 117 s, and 5.3 kJ per question on Jetson AGX Orin – more than 12× the cost of answering directly

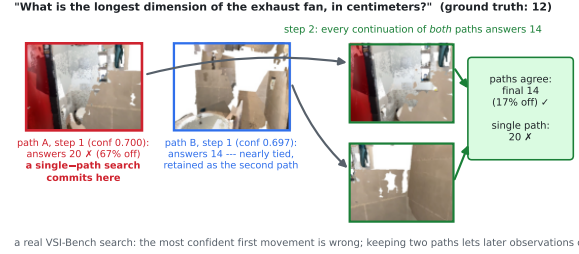


Figure 7: A single reasoning path fails on a real question: the most confident first movement (path A) answers 20 cm vs. ground truth 12, and confidence cannot separate it from the runner-up (0.700 vs. 0.697). Retaining two paths lets the next observations correct the error.

from the frames. And as the previous subsection showed, indiscriminate extra views can *lower* accuracy. Multi-path exploration therefore has to be conditional: created only when the input frames are insufficient (on our evaluation, 71% of questions need no exploration at all), pruned aggressively while exploring, and stopped as soon as independent paths agree. This is precisely the search that ViewTree’s confidence head and gate implement.

3 SYSTEM OVERVIEW

As shown in Figure 8, ViewTree contains three main components. First, confidence-guided viewpoint reasoning represents each reasoning path as a multi-step camera walk and retains several walks when the next useful movement is uncertain. Second, offline training teaches the view-conditioned answerer, camera policy, exploration gate, and confidence head. Third, the implementation reuses each scene reconstruction, caches views by camera pose, and batches rendering work. Sections 4–6 describe these components in the same order.

3.1 Online: Adaptive Viewpoint Reasoning

Let $V = \{v_1, \dots, v_n\}$ denote the available RGB frames and q a natural-language question. Consider a perspective-taking question whose answer is not visible from the current frames, such as asking which object is to the left of a chair when facing it from the opposite side. A useful reasoning path should first decide which change of viewpoint may reveal the relation, acquire that view, and then reconsider the question using the new observation and its camera pose. If the relation remains occluded, the reasoner should choose another movement from its new location. So the reasoning path therefore looks like

$$(q, V) \rightarrow a_1 \rightarrow (o_1, p_1) \rightarrow a_2 \rightarrow (o_2, p_2) \rightarrow y, \quad (1)$$

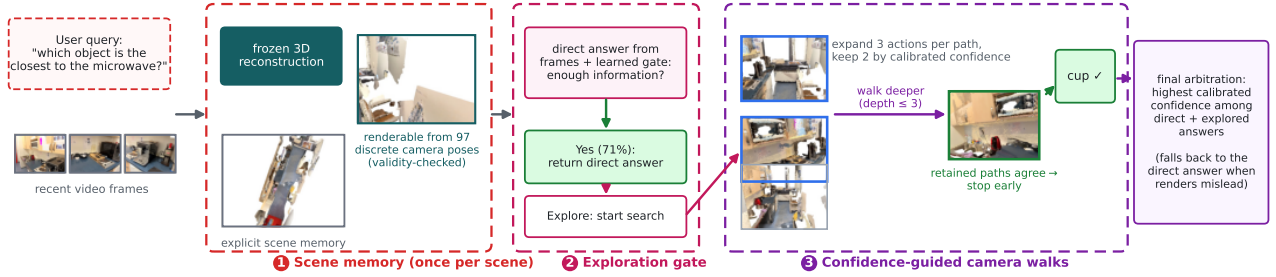


Figure 8: ViewTree overview on a real scene. The recent frames are reconstructed once into an explicit scene memory that can render validity-checked views from discrete camera poses (1); a learned gate answers directly when the frames suffice (2); otherwise confidence-guided camera walks acquire the missing views, stop early on agreement, and a final arbitration chooses between the direct and explored answers (3).

where every action a_i produces a novel view o_i at pose p_i and updates the answer y . For some questions, several first movements are plausible. Following only one can commit the reasoner to an uninformative side of the scene. The reasoning process should be able to retain several camera walks, evaluate each with its own observations, and spend later steps only on the walks that remain likely to answer correctly.

To realize this process, ViewTree first uses frozen VGGT to reconstruct the input frames into a stored 3D scene

$$\mathcal{M} = \Phi_{3D}(V), \quad (2)$$

where Φ_{3D} denotes the frozen reconstruction model and $\mathcal{M}$ contains scene points, camera parameters, and visibility information. For a camera pose p , the point-splat renderer $\mathcal{R}$ returns a 2D image o_p and a geometric support mask m_p :

$$(o_p, m_p) = \mathcal{R}(\mathcal{M}, p). \quad (3)$$

ViewTree begins by answering from the input frames and deciding whether exploration is necessary. If so, each active reasoning path proposes camera actions. Beam search expands the three actions with the largest controller probabilities and uses a calibrated confidence head to retain the two most promising resulting states. Each retained path keeps its own sequence of actions, poses, rendered observations, and answers. Search continues for at most three acquired views. It can stop earlier when the retained paths agree with sufficient confidence; otherwise, final arbitration compares the best explored state with the original direct answer. Thus, ViewTree implements the desired action-observation reasoning loop without requiring the VLM to process the complete 3D reconstruction. Section 4 gives the state, action, pruning, and stopping rules.

3.2 Offline: Reasoning Policy Training

ViewTree must learn four abilities: answer from the available views, decide whether another view is needed, choose where the camera should move, and estimate whether the current answer is reliable. We train these abilities in three SFT stages and a reinforcement-learning stage.

First, we train the VLM to answer from rendered views. The input contains the question, the original frames, and zero or more rendered views with their camera poses. Examples without rendered views teach the VLM to answer directly when the original frames are sufficient.

Second, we train the VLM to control the virtual camera. During training, the system tries camera movements that satisfy the physical constraints and checks whether the resulting views lead to the correct answer. It then selects the shortest successful viewpoint trajectory whose answer exceeds the required confidence margin. If the direct answer is already correct, the target action is STOP. This teaches the controller to avoid unnecessary exploration. We refer to this trial-and-check procedure as oracle search.

Third, we train the confidence head. Each partial viewpoint trajectory visited during oracle search is labelled according to whether the answer produced from that trajectory is correct. The confidence head learns to predict this outcome from the VLM’s internal state and is calibrated on held-out scenes. During inference, ViewTree uses this score to retain promising paths and remove weak ones. Search stops when the retained paths agree on an answer with higher confidence than the direct answer. Final arbitration then selects the highest-confidence answer among the direct and explored answers.

Finally, we optionally apply reinforcement learning over complete viewpoint trajectories while penalizing additional

camera steps. In our experiments, this stage improved answer prediction but did not improve camera-action selection. The final system therefore uses the controller trained from oracle trajectories.

3.3 System Execution

ViewTree repeatedly queries the same reconstructed scene while exploring different camera paths. To avoid repeated work, the system reconstructs each scene once and reuses the resulting world memory across questions and paths. Rendered views are cached by camera pose, and views requested at the same search level are rendered together. The current implementation processes the resulting VLM states sequentially.

ViewTree bounds the cost of camera exploration through the exploration gate, fixed search limits, and early stopping. It also removes camera actions whose target views are not sufficiently supported by the reconstruction. Section 6 presents the runtime design – reconstruction reuse, view and token caching, batched and decode-free search steps, and bounded search – together with its measured latency, energy, and memory on an embodied device.

4 ONLINE: ADAPTIVE VIEWPOINT REASONING

As described in Section 3.1, ViewTree organizes viewpoint acquisition as confidence-guided search over multi-step viewpoint trajectory. This section describes the reasoning state, valid camera actions, exploration gate, beam expansion, pruning, and stopping rules.

4.1 Viewpoint State and Action Space

Instead of providing the complete 3D scene memory to the VLM, the reasoning path in ViewTree contains only the information needed to select the next viewpoint and update the answer: the question, the original frames, the rendered views acquired so far, and a short textual pose tag for each view that tells the VLM where the camera was standing and which way it was facing when the view was captured. The complete 3D scene never enters the VLM’s input context, so the context length grows with the handful of acquired views rather than with the scene.

As shown in Figure 9, ViewTree exposes the scene memory through a small set of camera actions rather than free 3D coordinates: TURN-LEFT and TURN-RIGHT rotate the view in place, FORWARD moves to a nearby standing position along the current direction, NEXT-SPOT jumps to another valid standing position, LOOK-AROUND turns to the opposite direction, BIRD-EYE requests a single top-down view as the final acquisition, and STOP ends the trajectory. Discrete actions keep

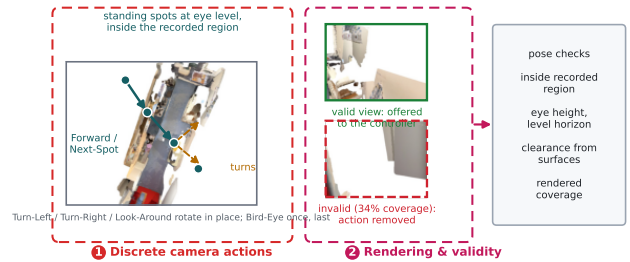


Figure 9: Camera actions and pose validity on a real reconstructed scene. The virtual camera moves between discrete standing spots and viewing directions (left); each candidate pose is rendered and checked before it is offered to the controller (middle, right).

three things simple at once: the training-time search over alternatives (Section 5) only needs to enumerate a handful of successors per state; the controller emits an action as a single token instead of numeric coordinates; and every pose maps to a small cache key, which the runtime exploits heavily (Section 6).

Before the controller selects an action, ViewTree checks whether the resulting pose is physically plausible and renderable: the pose must stay inside the region the camera actually recorded, keep the camera at eye height with a level horizon, maintain clearance from reconstructed surfaces, and produce a rendering with sufficient geometric support (Figure 9, right). Actions that fail these checks are removed from the controller’s choices. The controller therefore chooses only among views that can be reliably rendered, rather than having to learn to avoid unsupported poses from training examples.

4.2 Deciding When to Explore

Not every question requires a new viewpoint. When the original frames already contain the answer, rendering additional views adds unnecessary computation and may introduce irrelevant or incomplete visual information that reduces answer accuracy. ViewTree therefore first produces a direct answer from the original frames and, in the same breath, asks the model a single learned gating question: do these frames alone contain enough information to answer? If the gate answers YES, the direct answer is returned immediately after only two short VLM calls; across our evaluation this happens on 71% of questions. Only when the gate answers EXPLORE does ViewTree start the search: the camera controller scores the actions allowed at the current pose, and beam search creates alternative reasoning paths from the highest-scoring ones. The gate decides whether a search tree is needed at all, while the controller decides how each

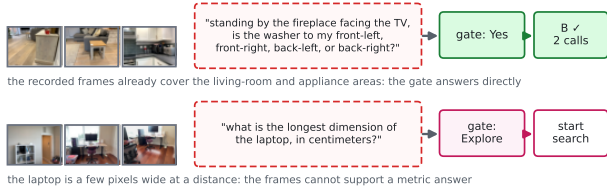


Figure 10: The exploration gate on two real questions. Top: the frames cover the queried areas, so the direct answer is returned after two calls. Bottom: the object’s size cannot be judged from distant frames, so the gate starts the search.

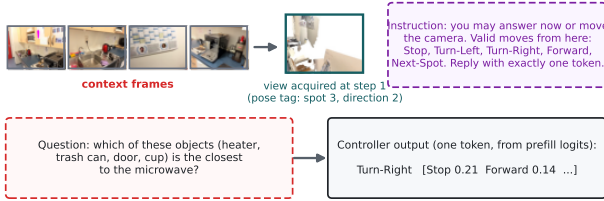


Figure 11: One camera-controller step. The valid moves are listed in the prompt, and the scores of all candidate actions are read from a single prefill pass, so a control step costs no answer decoding.

path should move through the scene. Figure 10 shows both outcomes on real questions.

4.3 Confidence-Guided Path Expansion and Pruning

Figure 11 shows one controller step. The controller receives the context frames, the views acquired so far with their pose tags, the question, and the list of currently valid moves, and replies with exactly one action token. ViewTree reads the scores of all valid actions directly from this single forward pass, expands the three highest-scoring actions, and creates a separate successor path for each: a movement action renders the new view and asks the VLM for an updated answer, while STOP marks the path as terminal without another render.

Each successor also needs a score that says how promising it is. Verbalized self-confidence and answer-token probability are known to be poorly calibrated for this purpose; instead, for each candidate path b , a small two-layer head reads the VLM’s last-token hidden state $\tilde{h}_b$ at the answer and predicts the probability that this answer is correct:

$$c_b = \sigma(\text{MLP}(\tilde{h}_b)/T), \quad (4)$$

where σ is the logistic function and the temperature T is fitted on held-out scenes. The head is trained on the outcomes of training-time search (Section 5), so its scores are

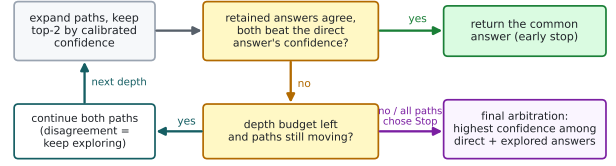


Figure 12: How a search ends: early stop on confident agreement, deeper exploration on disagreement, and final arbitration otherwise.

comparable across paths and depths. After expanding all active paths, ViewTree simply keeps the two successors with the highest calibrated confidence. Retained paths share the question and context frames, but their acquired views and later reasoning remain separate, so one early wrong turn cannot contaminate a sibling path.

4.4 Path Stopping and Final Answer Selection

Views are accumulated within each camera walk rather than combined across sibling paths: at every step, the VLM of a path sees the original frames followed by that path’s views in acquisition order, each preceded by its pose tag. The ordered tags tell the VLM how the camera moved, which lets a later step reuse evidence collected earlier along the same walk. Figure 13 traces a real ViewTree search: the retained paths approach the queried objects from different directions and converge on the same answer at depth three.

The search stops in one of three ways (Figure 12). If the two retained paths agree on the same answer and both are more confident than the direct answer, ViewTree stops early and returns the common answer – agreement between independently moving paths is strong evidence. If they disagree and depth remains, both paths continue so that their next camera actions can collect discriminating evidence. Otherwise, at the depth limit or when all retained paths choose STOP, ViewTree compares the calibrated confidence of the direct answer and of the retained explored answers, and returns the highest-scoring one:

$$\hat{y} = y_{k^*}, \quad k^* = \arg \max_{k \in \{0\} \cup \mathcal{B}_{\text{final}}} c_k, \quad (5)$$

where $\hat{y}$ is the returned answer, $k = 0$ denotes the direct-answer state, and $\mathcal{B}_{\text{final}}$ contains the retained explored states. This final arbitration matters in practice: it prevents an incomplete or distracting reconstruction from automatically overriding an answer that was already well supported by the original frames.

Algorithm 1 summarizes ViewTree inference.

With these settings, a gated question uses two VLM calls. Across the evaluation, ViewTree uses 4.5 calls per question

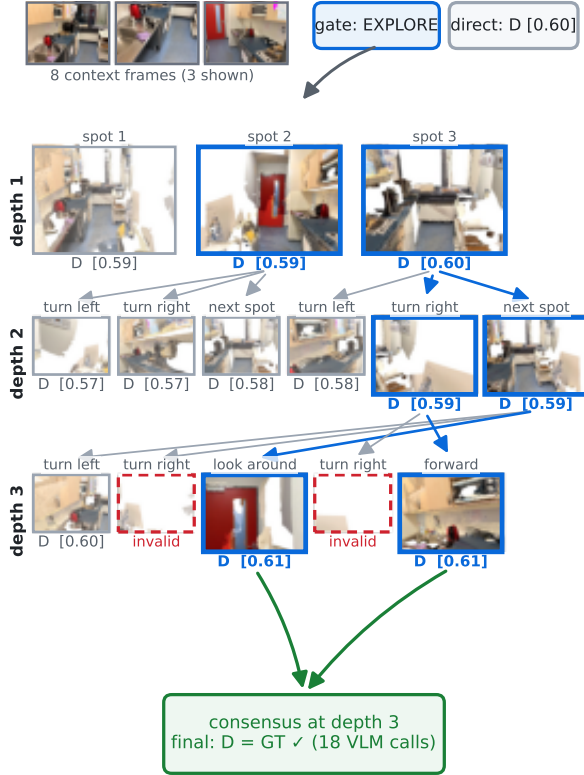


Figure 13: A real ViewTree search on a VSI-Bench question. The gate finds the frames insufficient; the beam explores three standing spots, prunes by calibrated confidence (blue borders), discards views without geometric support (red dashed), and stops at depth three when the retained paths agree on the correct answer.

on average; a full depth-three search can require approximately 20 answer and action-proposal calls, but such deep searches are rare (8% of questions). Because the gated calls are short, the common case is cheap: on the embodied device, the two-call gated route finishes faster than a single 16-frame direct-input call (Section 6). We therefore report calls, renders, and acquired views in addition to accuracy.

5 OFFLINE REASONING-POLICY TRAINING

ViewTree must learn four abilities: answer from the available views, decide whether another view is needed, select where the virtual camera should move, and estimate whether the current answer is reliable. As shown in Figure 14, we train these abilities in three stages, followed by an optional reinforcement-learning stage.

Algorithm 1 Confidence-guided ViewTree inference

Input: Question q , frames V , memory $\mathcal{M}$, depth budget D

Output: Answer y

```

1:  $(y_0, c_0, g) \leftarrow \text{DIRECTANSWERANDGATE}(q, V)$ 
2: if  $g = \text{YES}$  then
3:   return  $y_0$ 
4:  $\mathcal{B} \leftarrow \{\text{INIT}(q, V)\}$ 
5: for  $i = 0$  to  $D - 1$  do
6:    $\tilde{\mathcal{B}} \leftarrow \emptyset$ 
7:   for all  $b \in \mathcal{B}$  do
8:      $A_b \leftarrow \text{TOPVALIDACTIONS}(b, 3)$ 
9:     for all  $a \in A_b$  do
10:      if  $a = \text{STOP}$  then
11:         $\tilde{\mathcal{B}} \leftarrow \tilde{\mathcal{B}} \cup \{\text{TERMINAL}(b)\}$ 
12:      else
13:         $b' \leftarrow \text{OBSERVE}(b, a, \mathcal{M})$ 
14:         $\tilde{\mathcal{B}} \leftarrow \tilde{\mathcal{B}} \cup \{b'\}$ 
15:    $\mathcal{B} \leftarrow \text{TOPK}(\tilde{\mathcal{B}}, c, 2)$ 
16:   if  $\text{CONFIDENTAGREEMENT}(\mathcal{B}, c_0)$  then
17:     return common answer of  $\mathcal{B}$ 
18: return highest-confidence answer in  $\{y_0\} \cup \mathcal{B}$ 

```

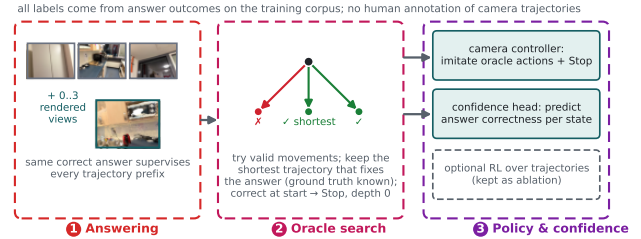


Figure 14: ViewTree training. The answerer is first taught to use rendered views (stage 1); a training-time oracle search discovers which camera trajectories fix wrong answers (stage 2); its actions and visited states then supervise the camera controller and the confidence head (stage 3).

We first teach the VLM to answer from the original frames and newly rendered views. We then use oracle search to produce camera-action and stopping targets. Finally, we use the states visited during oracle search to train the confidence head. The optional reinforcement-learning stage improves answer prediction but not camera control in our experiments; therefore, the final system uses the controller trained from oracle trajectories.

5.1 Learning to Answer from New Views

The VLM must answer using either the original frames alone or the original frames together with newly rendered views. We therefore train it with examples taken from different

steps of a camera trajectory. Each example contains the question, the original frames, the rendered views obtained so far, and the correct answer. Each rendered view is also paired with its camera location and viewing direction so that the VLM knows where the view was captured.

The VLM is trained with standard next-token prediction on the answer tokens, conditioned on the question, the original frames, and the rendered views obtained so far. The same correct answer supervises every prefix of a trajectory. Some training examples contain no rendered views; these teach the VLM to answer directly when the original frames provide enough information. Other examples contain one or more rendered views and teach the VLM to revise its answer when new information arrives.

5.2 Finding Useful Viewpoint Trajectories

The correct answer alone does not tell the controller where to move the virtual camera. We therefore use a training-time search procedure to find useful camera trajectories. The search starts from the original camera view and tries camera movements that satisfy the physical constraints. After each movement, the system renders the resulting view and asks the VLM to answer the question again. Because the correct answer is known during training, the system can check whether each camera trajectory helps the VLM produce the correct answer.

Concretely, an answer counts as correct when its evaluation score (exact match for multiple choice, relative accuracy for numerical answers) exceeds one half. If the VLM already answers correctly from the original frames, the selected trajectory is empty and the target action is STOP; these depth-zero examples are the most frequent labels and are what teach the controller not to over-explore. Otherwise, let $\mathcal{T}^+(q)$ contain the tested trajectories τ whose final answer is correct; the search keeps the shortest one, breaking ties by the answer margin $m(\tau)$, i.e., how strongly the VLM favors the correct answer at the end (Figure 14, middle):

$$\tau^* = \arg \max_{\tau \in \mathcal{T}^+(q), |\tau| = \min_{\tau' \in \mathcal{T}^+(q)} |\tau'|} m(\tau). \quad (6)$$

Preferring the shortest successful trajectory bakes the acquisition cost into the supervision itself: the controller is never shown a longer walk when a shorter one suffices. Figure 15 walks through one oracle trajectory.

The selected trajectory provides training examples for the camera controller. At each step, the views obtained so far form the controller input, and the next camera movement is used as the target action. After the final view, the target action is STOP. The search also keeps the states that produce incorrect answers, which are later used to train the confidence head.



the shortest trajectory whose answer becomes correct supervises the controller; every visited state -- including the wrong ones (35 cm) -- is labelled by answer correctness and trains the confidence head; questions that are already correct from the frames contribute depth-zero Stop labels, teaching the controller not to over-explore.

Figure 15: An oracle trajectory on a real training question. The direct answer is wrong; one FORWARD movement fixes it, so the controller’s targets are FORWARD then STOP, and every visited state is labelled by answer correctness for the confidence head.

5.3 Learning Exploration Policy and Confidence

The trajectories and states collected by oracle search supervise two modules at once: the camera controller, which decides where to move and when to stop, and the confidence head, which decides which paths to trust.

The selected oracle trajectory teaches the controller both where to move and when to stop: at every prefix h_i of the trajectory the controller imitates the next oracle action a_i^* , and at the final state it imitates STOP:

$$\mathcal{L}_{\text{ctrl}} = - \sum_{i=0}^{|\tau^*|} \log \pi_{\theta}(a_i^* | h_i), \quad a_{|\tau^*|}^* = \text{STOP}. \quad (7)$$

where π_{θ} is the controller’s distribution over camera actions. Invalid movements are removed before action prediction, so the controller only ever chooses among camera actions that can actually be executed from the current pose.

Before rendering a new view, ViewTree prompts the controller to choose between YES and EXPLORE. This decision reuses the controller’s learned stopping behavior and does not require a separate exploration classifier. YES returns the direct answer, whereas EXPLORE starts viewpoint-trajectory search. This initial decision determines whether another view is needed, while the subsequent controller decisions determine where the virtual camera should move.

During inference, ViewTree cannot compare a candidate answer with the ground truth. It must instead estimate which partial trajectory is likely to produce a correct answer. We therefore label every state visited during oracle search – including the many unsuccessful ones – by whether the answer produced at that state was correct, and train a small head on the VLM’s internal state to predict this outcome. The predicted probability is calibrated on held-out scenes before it is used by the search [12].

Unlike verbal confidence or answer-token probability, the calibrated score c_b (Eq. 4) is learned from the observed successes and failures of partial viewpoint trajectories. During inference, ViewTree uses it to rank candidate paths, retain

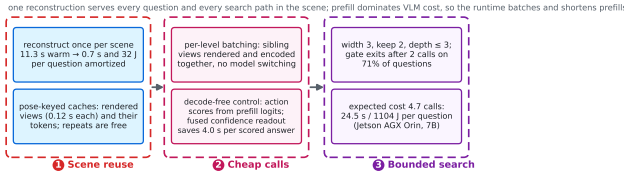


Figure 16: ViewTree runtime on the embodied device: the reconstruction is reused across questions and paths, repeated views are served from pose-keyed caches, control decisions decode nothing, and the search is bounded with a cheap gated exit.

the most promising paths, and remove weak paths. The search can stop when the retained paths agree on an answer whose confidence is higher than that of the direct answer. After exploration, ViewTree compares the direct and explored answers and returns the one with the highest calibrated confidence.

5.4 Reinforcement Learning

After supervised training, we optionally apply group-relative policy optimization to complete viewpoint trajectories [24]. The reward favors correct answers while penalizing additional or invalid camera movements. This objective encourages the model to improve its answer using as few rendered views as possible.

In our experiments, reinforcement learning improves answer prediction but does not improve camera-action selection. The final ViewTree system therefore uses the controller trained through imitation of the oracle trajectories. We retain the reinforcement-learned controller only as an ablation.

6 SYSTEM EXECUTION

ViewTree repeatedly queries the same reconstructed scene while exploring different camera paths. A direct implementation would reconstruct the scene for every question, render the same pose more than once, allow the number of candidate paths to grow with search depth, and pay a full VLM decoding pass for every control decision. Our runtime avoids all four costs (Figure 16). The numbers below are measured on Jetson AGX Orin (64 GB, JetPack 6, MAXN power mode) with the 7B backbone in bf16; Section 8 reports the other backbones.

Scene reuse and view caching. The scene memory is built once per scene and shared by every question and every search path: a 32-frame reconstruction takes 11.3 s warm on the device, which amortizes to 0.7 s and 32 J per question at the benchmark’s average of 18 questions per scene. Because camera poses are discrete (Section 4), every rendered view and its encoded visual tokens are cached under a small pose key;

a repeated request – common when sibling paths converge on the same viewpoint – skips both the renderer and the visual encoder. A cold render itself costs only 0.12 s, two orders of magnitude below a VLM call, so the pipeline’s cost is dominated by the VLM.

Batched, decode-free search steps. Views requested by sibling paths at the same search level are rendered in one pass and encoded as one batch, avoiding per-view model switching. Two further optimizations exploit how the VLM is invoked. First, the controller’s action choice is read from the logits of a single prefill pass over the listed valid moves (Figure 11), so a control step decodes nothing and costs the same 4.3 s as the short gate call. Second, the confidence head reads the VLM’s internal state directly from the answer call instead of issuing a separate scoring pass, which saves a measured 4.0 s per scored answer. On this hardware, prefill dominates VLM latency (about 1.9 s per thousand prompt tokens versus 0.11 s per decoded token), which is why ViewTree keeps its calls short – eight context frames plus a few renders – rather than feeding long frame sequences.

Bounded search under a device budget. The search uses fixed limits – expansion width three, two retained paths, depth at most three – and the gate exits after two short calls on 71% of questions. The expected cost at the measured route mix is 4.7 VLM calls, 24.5 s, and 1,104 J per question: about $2.1\times$ a single 16-frame direct-input call (11.7 s, 529 J), and $1.9\times$ cheaper than a depth-one tree that renders all five candidate views for most questions (46.3 s, 2,110 J). Notably, the 71%-frequency gated route alone (9.6 s) is cheaper than the single direct-input call, because two short-prefill calls beat one long-prefill call. The alternative of simply adding frames scales poorly: the direct-input baseline grows super-linearly to 300 s at 384 frames and exhausts the device memory at 512 frames, while ViewTree’s entire adaptive exploration costs roughly what five extra context frames would. Peak memory with both the VLM and the reconstruction model resident is 25.9 GB, leaving ample headroom on the 64 GB device for the 7B backbone; the 32B backbone fits with 4-bit weights.

7 EXPERIMENT SETTINGS

We evaluate ViewTree both on a server testbed and on an embodied AI device. All controller training and large-scale benchmark evaluations run on a server with $8\times$ NVIDIA H100 GPUs; the complete inference pipeline is additionally deployed on an NVIDIA Jetson AGX Orin (64 GB) to characterize on-device feasibility. We apply VLM backbones with various parameter sizes to match different device budgets: Qwen2.5-VL-3B, Qwen2.5-VL-7B, and Qwen2.5-VL-32B [3], where the 32B model is deployed on Jetson with 4-bit weight quantization.

System setup. On the server, all models run in PyTorch with bfloat16 precision. On Jetson AGX Orin, the VLM runs with quantized weights, and the remaining pipeline stages are executed as stage-level batches: the frozen VGGT [30] reconstruction is computed once per scene and reused across all questions and reasoning branches; candidate views proposed at the same tree level are rendered in one point-splat pass and encoded as one batch; rendered views and their encoded tokens are cached by camera pose and scene version, so repeated requests skip both the renderer and the visual encoder. The end-to-end latency reported in Section 8 includes reconstruction (amortized and upfront), rendering, visual encoding, and all VLM calls of the reasoning tree.

Benchmarks. Our experiments evaluate the spatial-cognition capabilities that ViewTree is designed to improve. Following the task grouping of Section 7, we use *Metric Measurement* (absolute distance, object size, room size), *Perspective-dependent Reasoning* (relative direction, relative distance, object counting, route planning), and *Spatiotemporal Tracking* (appearance order, camera-motion and camera-object relations). We evaluate on four benchmark datasets:

- **VSI-Bench** [33] contains >5,000 QA pairs from 288 indoor videos over 10 spatial tasks; we report absolute distance (A.D.), object size (O.S.), relative distance (R.Dt.), relative direction (R.Dr.), object count (O.C.), and appearance order (Ap.Or.). To prevent any information leakage from confidence-head fitting, all VSI-Bench numbers are reported on a held-out half of the scenes (144 scenes, 2,557 questions) that is never touched by any training or calibration; splits are at scene level throughout.
- **STI-Bench** [17] contains 300 videos and >2,000 QA pairs; we report dimensional measurement (D.Meas.) and 3D video grounding (Grd.) to complement VSI-Bench.
- **VSTI-Bench** [10] contains 5,736 QA pairs on ScanNet validation videos over 9 spatio-temporal types; we report the camera-object relation (C.O.) and object-object relation (O.O.) groups.
- **OST-Bench** [18] contains 5,557 multiple-choice questions probing online spatio-temporal reasoning over an agent’s cumulative image history; we report the overall accuracy (Ovr.).

QA performance is evaluated as the percentage of correct answer choices; numerical tasks (absolute distance, object size, room size, and the numerical VSTI types) use Mean Relative Accuracy.

Baselines. We compare ViewTree against the following baselines, all using the same VLM backbone, visual resolution, frame budgets, and answer scorer:

- **Direct input** uniformly downsamples the video and feeds the frames directly to the VLM.
- **Standard SFT** fine-tunes the backbone on the training QA with answer supervision only, using the input frames without any rendered view.
- **Standard SFT+GRPO** further applies GRPO [24] with answer-correctness reward on top of standard SFT.
- **Video-CoT** prompts the same backbone to produce step-by-step textual reasoning over the input frames before committing to an answer.
- **Static scene memory** attaches a fixed set of rendered views (a bird’s-eye view plus eye-level renders) to the input frames of every question, i.e., scene reconstruction without adaptive exploration.

Training setup. All trained components are LoRA adapters [13] (rank 16) on the frozen backbone plus the two-layer confidence head; the backbone weights are never updated. Training data come from the VLM-3R spatial-QA corpus [10] (493,663 QA pairs over 1,709 ScanNet [8] and ScanNet++ [35] scenes); every scene that appears in any evaluation benchmark is excluded, and all fine-tuned baselines use the same corpus and compute budget. The answerer is trained on roughly 100k walk-generated examples (Section 5), the controller and confidence head on 8,639 oracle walks, and the optional RL stage runs GRPO with an answer-correctness reward over 30k walk episodes.

8 PERFORMANCE EVALUATION

8.1 Spatial Task Performance

Table 1 reports the full benchmark grid with the 7B backbone, and Table 2 the backbone-scale study on VSI-Bench; key deltas are given with scene-bootstrap 95% confidence intervals.

Main results (7B). On VSI-Bench, ViewTree reaches 0.530 average over the ten task types, against 0.313 for direct frame input. Most of this gap is closed by training: the standard-SFT baseline, fine-tuned on the same corpus, reaches 0.509 without any rendered view. On top of this strongest baseline, ViewTree’s adaptive exploration adds +2.1 points [−0.0, +4.3], and the per-type columns show exactly where: relative direction (0.467 → 0.516, with the hard split +6.2), relative distance (+5.0), and appearance order (+4.6) – the perspective-dependent questions where a new viewpoint changes the visible geometry. On object counting and room size the extra renders *reduce* accuracy slightly (−1.4 and −2.2): those types are answered best from the original frames, which is why the arbitration of Section 4 retains the direct answer when exploration does not raise confidence. The SFT+GRPO row confirms that answer-reward RL improves the answer

distribution (0.534 single pass, +2.5 [+0.3, +5.0] over standard SFT) but, as Section 8.4 shows, it does not improve the camera policy itself.

Cross-benchmark transfer. The remaining columns give an honest picture of where view exploration pays. On VSTI-Bench, whose camera-object and object-object questions match the training templates, ViewTree is +19.4 (C.O.) and +17.4 (O.O.) points over direct input, but its margin over the standard-SFT single pass is not significant ($-0.5 [-1.2, +0.2]$ on the 9-type mean): template-matched training data does the work. On STI-Bench, every fine-tuned system sits below the zero-shot backbone (ViewTree $-2.6 [-4.4, -0.7]$ on the 8-task mean), because half of STI’s tasks come from outdoor driving and object-pose domains that an indoor corpus cannot teach. On OST-Bench the walk actively hurts ($-2.1 [-3.1, -1.2]$ versus its own single pass): OST asks about the agent’s own trajectory and what it has seen, and a consensus among rendered views of the *current* reconstruction overrides direct answers that were more often right. Multi-step view acquisition earns its cost only where the question concerns room geometry that the input frames under-cover; a deployment should gate the walk per question family, which the current gate learns only for VSI-style questions.

Backbone scale (Table 2). The 32B backbone is the strongest zero-shot model (0.380) but every 7B-tuned recipe fails to transfer to it: benchmark-QA SFT *lowers* its accuracy by 2.9 points [$-4.9, -0.7$], and the training-free ViewTree variant that gains +2.8 [+1.0, +4.7] at 7B loses $-3.2 [-5.2, -1.1]$ at 32B, where token-confidence pruning lets render-limited consensus override strong direct answers. At the other end, the corpus moves the mobile-class 3B backbone from 0.321 to 0.497 with only a 30k-example subset – statistically at the level of the 7B data-matched baseline (0.509) – showing that the answering skill lives in the data rather than the capacity, while the transplanted training-free tree is neutral there (-0.7). Adaptive exploration therefore requires per-backbone calibration of the gate and confidence head; the full trained system is built at 7B in this paper.

8.2 On-Device Compute Efficiency

Figure 17 presents the measured per-question cost of ViewTree and the baselines on Jetson AGX Orin, using the deployment of Section 6. At the evaluation’s route mix, ViewTree costs 24.5 s and 1.1 kJ per question – about $2.1\times$ a single 16-frame call (11.7 s, 0.53 kJ) and $1.9\times$ *cheaper* than the non-adaptive depth-one tree that renders all five candidate views for most questions (46.3 s, 2.1 kJ). The route mix in Figure 17(b) explains why: the exploration gate answers 71% of questions after two short calls in 9.6 s – *less* than the direct-input baseline, because two short-prefill calls are cheaper than one

long-prefill call over 16 frames – and the expensive depth-2/3 walks (71–117 s) are taken on only 14% of questions. Figure 17(c) decomposes the routes: VLM calls dominate, rendering and encoding the requested views costs 0.6 s per level, and the per-scene reconstruction (11.3 s warm) amortizes to 0.7 s and 32 J per question at the benchmark’s 18 questions per scene. Peak memory with both the VLM and the reconstruction model resident is 25.9 GB, leaving ample headroom on the 64 GB device; the 32B backbone fits with 4-bit weights.

8.3 Performance over Different Levels of Scene Complexities

Figure 18 analyzes how scene complexity affects accuracy, using the 7B backbone on the VSI-Bench held-out split. Direct frame input degrades on every complexity axis: from 0.34 to 0.26 as rooms grow from under 14 m² to over 23 m², and from 0.33 to 0.28 as the walkthrough video grows beyond 96 seconds – with a fixed frame budget, larger scenes and longer videos leave more of the room uncovered by the sampled frames. ViewTree is the most robust system on both axes: its score *rises* with video duration (0.52 $\rightarrow$ 0.55), and its margin over the standard-SFT baseline grows exactly where coverage thins – from -0.8 points on short videos to +2.9 on long ones, and from 0.0 in small rooms to +2.5 in mid-size and large rooms – because a question about an under-covered region can be answered by walking the virtual camera there instead of hoping a sampled frame saw it. Object counting shows the opposite regime: fine-tuning dominates (direct input collapses from 0.36 to 0.06 as the count grows, while both trained systems stay near 0.32–0.35), and the rendered views add nothing for small counts; ViewTree’s arbitration keeps it within 2.5 points of the frames-only baseline while gaining a small edge on scenes with five or more target objects.

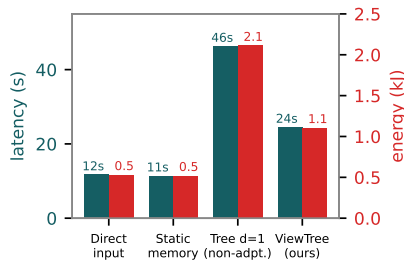
8.4 Ablation Studies

View-acquisition strategies. Table 3 (left) compares confidence-guided beam search against simpler acquisition strategies with the same trained answerer, on the VSI-Bench held-out split: presenting all five candidate views unconditionally (static memory), a random walk with the same depth budget and pruning, and a greedy walk (beam width 1). Numbers for the alternative strategies are being finalized.

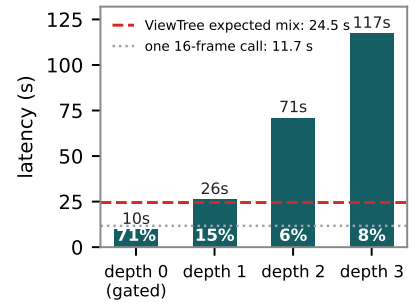
Transplanting the recipe across backbones. Table 3 (right) applies the training-free variant of the same search (token-confidence pruning, identical pose constraints) to all three backbones. It helps only the 7B backbone it was tuned on (+2.8); it is neutral at 3B, where the small model answers weakly from renders, and *harmful* at 32B (-3.2), where the stronger model’s direct answers are overridden by render-limited consensus (the gate releases only 17% of questions

Method (Qwen2.5-VL-7B)	VSI-Bench (held-out)							STI-Bench			VSTI-Bench		OST
	A.D.	O.S.	R.Dt.	R.Dr.	O.C.	Ap.Or.	Avg	D.Meas.	Grd.	Avg	C.O.	O.O.	Ovr.
Direct input	0.088	0.340	0.391	0.367	0.254	0.289	0.313	0.301	0.315	0.371	0.530	0.646	0.540
Video-CoT	–	–	–	–	–	–	–	–	–	–	–	–	–
Standard SFT	0.361	0.649	0.515	0.467	0.667	0.628	0.509	0.318	0.344	0.343	0.716	0.826	0.516
Standard SFT+GRPO	0.404	0.672	0.529	0.489	0.664	0.623	0.534	–	–	–	–	–	–
Static scene memory	–	–	–	–	–	–	–	–	–	–	–	–	–
ViewTree	0.346	0.652	0.565	0.516	0.653	0.674	0.530	0.298	0.312	0.345	0.724	0.820	0.496

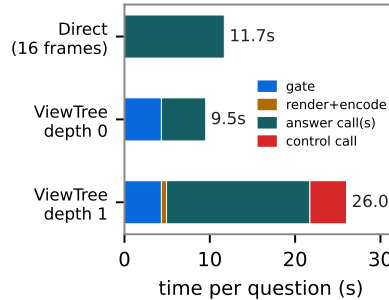
Table 1: Performance with Qwen2.5-VL-7B. Task acronyms are defined in Section 7; VSI Avg is the mean over all ten VSI task types, STI Avg over all eight STI tasks. R.Dr. averages the easy/medium/hard splits. Static scene memory uses the walk-trained answerer with all five rendered views attached to every question. “–”: prompting- and render-exposure baselines are compared on VSI-Bench.



(a) Per-question cost by method



(b) Adaptive route mix



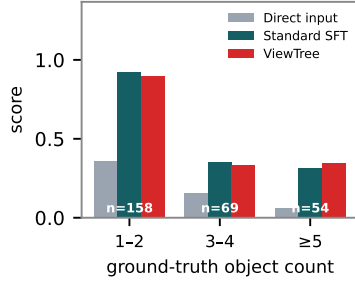
(c) Where the time goes

Figure 17: Measured compute efficiency on Jetson AGX Orin (7B backbone; energy integrates the three INA3221 power rails).

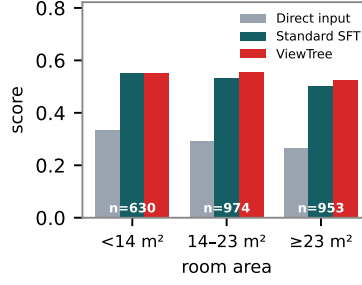
for direct answering there). Adaptivity itself is not free across model sizes: the gate and confidence head must be calibrated per backbone.

Component ablations. Table 4 isolates the contribution of each part of ViewTree. Capping the search at depth one costs 1.3 points despite *more* VLM calls (up to 8 versus 4.5), because the gate of the full system answers 71% of questions directly while the depth-one tree renders five views for most of them; the extra depth pays on the 209 questions that reach consensus only at depth two or three. Removing exploration entirely (the walk-trained answerer on frames alone) costs

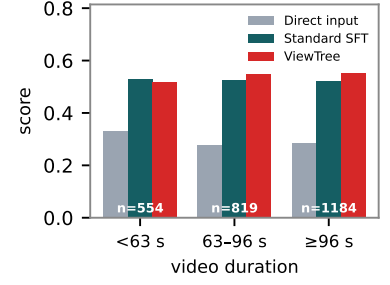
0.6 points but is the cheapest option at one call – this gap, concentrated on relative direction and distance, is what the walk buys. Replacing the walk-trained answerer with the plain corpus SFT costs a further 1.5 points, showing that training with rendered context improves even frames-only answering. Finally, adding GRPO to the controller does not help (0.522, -0.8 $[-2.2, +0.7]$ versus the imitation-trained controller): the policy collapsed toward stopping early during RL, and the same adapter answering frames-only scores 0.534 – the reward moved the answer tokens, not the camera



(a) Object Number



(b) Spatial Scale



(c) Temporal Duration

Figure 18: Spatial task performance under different levels of scene complexity (VSI-Bench held-out split, 7B backbone): counting questions binned by ground-truth object count; all questions binned by the scene’s room area and by video duration (tercile splits).

Method	A.D.	O.S.	R.Dt.	R.Dr.	O.C.	Ap.Or.	Avg
<i>Qwen2.5-VL-3B</i>							
Direct input	0.183	0.196	0.342	0.484	0.304	0.184	0.321
Video-CoT	–	–	–	–	–	–	–
Benchmark-QA SFT	0.223	0.173	0.344	0.448	0.228	0.176	0.307
Benchmark-QA SFT+GRPO	0.230	0.187	0.347	0.445	0.232	0.159	0.306
Standard SFT (corpus, 30k)	0.358	0.611	0.534	0.473	0.669	0.623	0.497
Static scene memory	–	–	–	–	–	–	–
ViewTree (training-free)	0.144	0.205	0.366	0.465	0.275	0.230	0.314
<i>Qwen2.5-VL-32B</i>							
Direct input	0.257	0.500	0.501	0.380	0.286	0.305	0.380
Video-CoT	–	–	–	–	–	–	–
Benchmark-QA SFT	0.257	0.529	0.501	0.293	0.343	0.293	0.351
Static scene memory	–	–	–	–	–	–	–
ViewTree (training-free)	0.219	0.472	0.457	0.329	0.270	0.356	0.348

Table 2: Backbone-scale study on the VSI-Bench held-out split. The trained ViewTree controller and confidence head are 7B-specific, so 3B/32B use the training-free variant (token-confidence pruning, human-constrained poses). Benchmark-QA SFT/GRPO train on MindCube-style QA [36]; the 30k-subset corpus SFT is the data-matched answerer at 3B (the full corpus SFT does not fit 32B fine-tuning in our budget); static scene memory is zero-shot render exposure. GRPO at 32B was not run.

Strategy (7B)	Avg	Calls	Backbone	Direct	+Tree
All views (static)	–	1	3B	0.321	0.314
Random walk	–	–	7B	0.313	0.341
Greedy walk (b=1)	–	–	32B	0.380	0.348
Ours (b=3, keep 2)	0.530	4.5			

Table 3: Left: view-acquisition strategies with the same trained answerer (VSI-Bench held-out, average over ten types and VLM calls per question). Right: the training-free tree transplanted across backbone sizes.

policy, so the deployed system uses the imitation-trained controller without RL.

Variant	Avg	Calls
ViewTree (full)	0.530	4.5
depth capped at 1	0.517	≤8
w/o exploration (frames only)	0.524	1
w/o walk training (corpus SFT)	0.509	1
w/ RL controller (GRPO over walks)	0.522	4.6

Table 4: Component ablations on the VSI-Bench held-out split (7B): average over the ten task types and VLM calls per question.

REFERENCES

- [1] Kittipat Apicharttrisor, Xukan Ran, Jiasi Chen, Srikanth V. Krishnamurthy, and Amit K. Roy-Chowdhury. 2019. Frugal Following: Power Thrifty Object Detection and Tracking for Mobile Augmented Reality. In *Proceedings of ACM SenSys*.
- [2] Daichi Azuma, Taiki Miyashishi, Shuhei Kurita, and Motoaki Kawanabe. 2021. ScanQA: 3D Question Answering for Spatial Scene Understanding. *arXiv preprint arXiv:2112.10482* (2021).
- [3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923* (2025).
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv preprint arXiv:2307.15818* (2023).
- [5] Meng Cao, Xingyu Li, Xue Liu, Ian Reid, and Xiaodan Liang. 2025. SpatialDreamer: Incentivizing Spatial Reasoning via Active Mental Imagery. *arXiv preprint arXiv:2512.07733* (2025).
- [6] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. 2024. SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14455–14465.
- [7] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. 2024. YOLO-World: Real-Time Open-Vocabulary Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [8] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. 2017. ScanNet: Richly-Annotated 3D Reconstructions of Indoor Scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

- [9] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, et al. 2023. PaLM-E: An Embodied Multimodal Language Model. In *International Conference on Machine Learning (ICML)*.
- [10] Zhiwen Fan, Jian Zhang, Renjie Li, Junge Zhang, Runjin Chen, Hezhen Hu, Kevin Wang, Huaizhi Qu, Shijie Zhou, Dilin Wang, et al. 2025. VLM-3R: Vision-Language Models Augmented with Instruction-Aligned 3D Reconstruction. *arXiv preprint arXiv:2505.20279* (2025).
- [11] Xingyu Fu, Yushi Hu, Bangzheng Li, Yu Feng, Haoyu Wang, Xudong Lin, Dan Roth, Noah A. Smith, Wei-Chiu Ma, and Ranjay Krishna. 2024. BLINK: Multimodal Large Language Models Can See but Not Perceive. In *European Conference on Computer Vision*. 148–166.
- [12] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *International Conference on Machine Learning*. 1321–1330.
- [13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [14] Saurav Jha, M. Jehanzeb Mirza, Wei Lin, Shiqi Yang, and Sarath Chandar. 2025. Probing the Effectiveness of World Models for Spatial Reasoning through Test-Time Scaling. *arXiv preprint arXiv:2512.05809* (2025).
- [15] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, et al. 2024. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246* (2024).
- [16] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Roland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. 2023. Segment Anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [17] Yun Li, Yiming Zhang, Tao Lin, Xiangrui Liu, Wenxiao Cai, Zheng Liu, and Bo Zhao. 2025. STI-Bench: Are MLLMs Ready for Precise Spatial-Temporal World Understanding? *arXiv preprint arXiv:2503.23765* (2025).
- [18] Jingli Lin, Chenming Zhu, Runsen Xu, Xiaohan Mao, Xihui Liu, Tai Wang, and Jiangmiao Pang. 2025. OST-Bench: Evaluating the Capabilities of MLLMs in Online Spatio-temporal Scene Understanding. *arXiv preprint arXiv:2507.07984* (2025).
- [19] Luyang Liu, Hongyu Li, and Marco Gruteser. 2019. Edge Assisted Real-Time Object Detection for Mobile Augmented Reality. In *Proceedings of ACM MobiCom*.
- [20] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, et al. 2024. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. In *European Conference on Computer Vision (ECCV)*.
- [21] Xiaojian Ma, Silong Yong, Zilong Zheng, Qing Li, Yitao Liang, Song-Chun Zhu, and Siyuan Huang. 2022. SQA3D: Situated Question Answering in 3D Scenes. *arXiv preprint arXiv:2210.07474* (2022).
- [22] Arjun Majumdar, Anurag Ajay, Xiaohan Zhang, Pranav Putta, Sriram Yenamandra, Mikael Henaff, Sneha Silwal, Paul McVay, Oleksandr Maksymets, Sergio Arnaud, Karmesh Yadav, et al. 2024. OpenEQA: Embodied Question Answering in the Era of Foundation Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16488–16498.
- [23] Zhangyang Qi, Zhixiong Zhang, Ye Fang, Jiaqi Wang, and Hengshuang Zhao. 2025. GPT4Scene: Understand 3D Scenes from Videos with Vision-Language Models. *arXiv preprint arXiv:2501.01428* (2025).
- [24] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300* (2024).
- [25] Nathaniel Simon and Anirudha Majumdar. 2023. MonoNav: MAV Navigation via Monocular Depth Estimation and Reconstruction. In *International Symposium on Experimental Robotics (ISER)*.
- [26] Fei Wang, Xingyu Fu, James Y. Huang, Zekun Li, Qin Liu, Xiaogeng Liu, Mingyu Derek Ma, Nan Xu, Wenxuan Zhou, Kai Zhang, et al. 2024. MuirBench: A Comprehensive Benchmark for Robust Multi-image Understanding. *arXiv preprint arXiv:2406.09411* (2024).
- [27] Fengyun Wang, Sicheng Yu, Jiawei Wu, Jinhui Tang, Hanwang Zhang, and Qianru Sun. 2025. 3D Question Answering via only 2D Vision-Language Models. *arXiv preprint arXiv:2505.22143* (2025).
- [28] Haoming Wang, Qiyao Xue, Weichen Liu, and Wei Gao. 2026. MosaicThinker: On-Device Visual Spatial Reasoning for Embodied AI via Iterative Construction of Space Representation. *arXiv preprint arXiv:2602.07082* (2026).
- [29] Haochen Wang, Yucheng Zhao, Tiancai Wang, Haoqiang Fan, Xiangyu Zhang, and Zhaoxiang Zhang. 2025. Ross3D: Reconstructive Visual Instruction Tuning with 3D-Awareness. *arXiv preprint arXiv:2504.01901* (2025).
- [30] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. 2025. VGGT: Visual Geometry Grounded Transformer. *arXiv preprint arXiv:2503.11651* (2025).
- [31] Xubin Wang, Zhiqing Tang, Jianxiong Guo, Tianhui Meng, Chenhao Wang, Tian Wang, and Weijia Jia. 2025. Empowering Edge Intelligence: A Comprehensive Survey on On-Device AI Models. *Comput. Surveys* 57, 9 (2025), 1–39.
- [32] Yikun Wang, Siyin Wang, Qinyuan Cheng, Zhaoye Fei, Liang Ding, Qipeng Guo, Dacheng Tao, and Xipeng Qiu. 2025. VisuoThink: Empowering LVLM Reasoning with Multimodal Tree Search. *arXiv preprint arXiv:2504.09130* (2025).
- [33] Jihan Yang, Shusheng Yang, Anjali W. Gupta, Rilyn Han, Fei-Fei Li, and Saining Xie. 2024. Thinking in Space: How Multimodal Large Language Models See, Remember, and Recall Spaces. *arXiv preprint arXiv:2412.14171* (2024).
- [34] Yuncong Yang, Jiageng Liu, Zheyuan Zhang, Siyuan Zhou, Reuben Tan, Jianwei Yang, Yilun Du, and Chuang Gan. 2025. MindJourney: Test-Time Scaling with World Models for Spatial Reasoning. *arXiv preprint arXiv:2507.12508* (2025).
- [35] Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. 2023. ScanNet++: A High-Fidelity Dataset of 3D Indoor Scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [36] Baiqiao Yin, Qineng Wang, Pingyue Zhang, Jianshu Zhang, Kangrui Wang, Zihan Wang, Jieyu Zhang, Keshigeyan Chandrasegaran, Han Liu, Ranjay Krishna, Saining Xie, Manling Li, Jiajun Wu, and Fei-Fei Li. 2025. Spatial Mental Modeling from Limited Views. *arXiv preprint arXiv:2506.21458* (2025).
- [37] Shoubin Yu, Yue Zhang, Zun Wang, Jaehong Yoon, Huaxiu Yao, Mingyu Ding, and Mohit Bansal. 2026. When and How Much to Imagine: Adaptive Test-Time Scaling with World Models for Visual Spatial Reasoning. *arXiv preprint arXiv:2602.08236* (2026).
- [38] Zaibin Zhang, Yuhao Wu, Lianjie Jia, Yifan Wang, Zhongbo Zhang, Yijiang Li, Binghao Ran, Fuxi Zhang, Zhuohan Sun, Zhenfei Yin, Lijun Wang, and Huchuan Lu. 2026. Think3D: Thinking with Space for Spatial Reasoning. *arXiv preprint arXiv:2601.13029* (2026).
- [39] Haoyu Zhao, Akide Liu, Zeyu Zhang, Weijie Wang, Feng Chen, Ruihan Zhu, Gholamreza Haffari, and Bohan Zhuang. 2026. CoV: Chain-of-View Prompting for Spatial Reasoning. *arXiv preprint arXiv:2601.05172* (2026).

- [40] Jensen Zhou, Hang Gao, Vikram Voleti, et al. 2025. Stable Virtual Camera: Generative View Synthesis with Diffusion Models. *arXiv preprint arXiv:2503.14489* (2025).
- [41] Chenming Zhu, Jingli Lin, Yilin Long, Peizhou Cao, Tai Wang, Jiangmiao Pang, and Xihui Liu. 2026. Thinking with Imagination: Agentic Visual Spatial Reasoning with World Simulators. *arXiv preprint arXiv:2606.06476* (2026).